MATLAB SCRIPT DIGITAL FILTER +FFT simulations

```
% 1. System Parameters
R = 50;
M = 1;
N = 2;
Fs_in = 2400000; % 2.4 MHz input sample rate
fs_audio = 48000; % 48 kHz audio sample rate


% 2. Create the CIC Filter Object
hcic = dsp.CICDecimator(R, M, N);


% 3. Extract Pure Mathematical Frequency Response Data
[h, w] = freqz(hcic, 8192*4);


% 4. Convert X-Axis to Real Frequency (kHz) and Normalized Scale
f_Hz = (w / pi) * (Fs_in / 2);
f_kHz = f_Hz / 1000;
f_norm = f_Hz / (Fs_in / 2); % Normalization range from 0 to 1
(relative to Nyquist)







% 5. Calculate Y-Axis in Decibels (dB)
mag_dB = 20*log10(abs(h)); % Raw magnitude response in dB


% 6. Normalize Amplitude to 0 dB Peak
mag_dB_norm = mag_dB - max(mag_dB);


figure('Name', 'Risposta CIC Definitiva', 'Color', 'w');
plot(f_kHz, mag_dB_norm, 'g', 'LineWidth', 1.5,'DisplayName', 'Solo
CIC'); % Green line plot
grid on;
set(gca, 'GridColor', [0.8 0.8 0.8], 'GridAlpha', 0.5); % Clean and
elegant grid
set(gca, 'Color', 'w'); % White plot background
```

```
set(gca, 'XColor', 'k', 'YColor', 'k'); % Black axis lines and labels
set(gca, 'FontWeight', 'bold', 'FontSize', 11); % Bold and larger
axis numbers
set(gca, 'GridColor', 'k', 'GridAlpha', 0.15); % Soft black grid
lines
xlim([0 500]);
ylim([-160 5]);




xlabel('Frequency (kHz)', 'FontSize', 12, 'Color', 'k', 'FontWeight',
'bold');
ylabel('Magnitude Normalized (dB)', 'FontSize', 12, 'Color', 'k',
'FontWeight', 'bold');
title('Frequency response CIC (N=2, R=50 ,M=1)', 'FontSize', 12,
'Color', 'k', 'FontWeight', 'bold');


% 10. Red reference lines for Nulls and Cutoff frequencies
xline(48, 'r--', 'Primo Nullo (48 kHz)', 'LabelVerticalAlignment',
'bottom', 'FontWeight', 'bold');
%xline(20, 'r--', 'l', 'LabelVerticalAlignment', 'bottom',
'FontWeight', 'bold');
%yline(-5.27781246, 'r--', 'Attenuazione a 20khz circa non si trova
perfettamnte il punto esatto', 'LineWidth', 1.5);
yline(-3,'r--','Attenuation at -3dB  is near
-15.28KHz','LineWidth',1.5);
xline(15.28,'r--','Cutoff frequency at -3dB  is near
15.28KHz','LineWidth',1.5);




% ========================================================
% INTERACTIVE VISUALIZATION USING FVTOOL
% ========================================================


% 1. Open FVTool
hfvt = fvtool(hcic, 'Fs', Fs_in);


% 2. Force real frequency view (Turn off normalized frequency)
```

```
hfvt.NormalizedFrequency = 'off'; % Set to 'on' to normalize
frequency relative to Fs_in/2 (1.2 MHz Nyquist)


% 3. Automatically normalize magnitude to 0 dB peak
hfvt.NormalizeMagnitudeto1 = 'on';


hfvt.FrequencyRange = 'Specify freq. vector';
hfvt.FrequencyVector = linspace(0, 300000, 10000);




% 5. Aesthetics: White background and green response curve
set(hfvt, 'Color', 'w');
hlines = findobj(hfvt, 'Type', 'line');
set(hlines, 'Color', 'g', 'LineWidth', 1.5);
ax_fvtool = get(hfvt, 'CurrentAxes');


% 7. Draw the reference line for the first null (48 kHz)
xline(ax_fvtool, 48, 'r--', 'Primo Nullo (48 kHz)',
'LabelVerticalAlignment', 'bottom', 'FontWeight', 'bold',
'FontSize', 11);


% 1. Compensation FIR Filter Parameters and Design
CICCompDecim = dsp.CICCompensationDecimator( ...
    'DecimationFactor', 1, ...
    'CICRateChangeFactor', 50, ...
    'CICNumSections', 2, ...
    'CICDifferentialDelay', 1, ...
    'SampleRate', 48000, ...
    'PassbandFrequency', 20000, ...
    'StopbandFrequency', 24000, ...
    'StopbandAttenuation', 60);


% 2. Coefficient Extraction
c = coeffs(CICCompDecim);
fir_coeffs = c.Numerator; % These are the real FIR filter tap values
```

```
%%%%%
% Display the impulse response of the compensating FIR filter in the
time domain
impz(CICCompDecim);


% === Bold Text Styling for the Impulse Response Plot ===
ax_impz = gca; % Capture the current axes handles generated by impz
set(ax_impz, 'Color', 'w'); % White internal plot background
set(ax_impz, 'XColor', 'k', 'YColor', 'k'); % Solid black axis lines
set(ax_impz, 'FontSize', 11, 'FontWeight', 'bold'); % Bold fonts for
axis ticks


% Apply bold styling to auto-generated titles and labels
title(ax_impz, 'Compensating FIR filter response in time domain',
'FontSize', 12, 'FontWeight', 'bold', 'Color', 'k');
xlabel(ax_impz, 'n (Samples)', 'FontSize', 12, 'FontWeight', 'bold',
'Color', 'k');
ylabel(ax_impz, 'Amplitude', 'FontSize', 12, 'FontWeight', 'bold',
'Color', 'k');


% Optional: Increase line width and marker size for stems
h_stems = findobj(ax_impz, 'Type', 'stem');
set(h_stems, 'LineWidth', 1.5, 'MarkerSize', 6);




%%%%%%%%


% 3. Print Coefficients (Useful for thesis documentation, C code or
FPGA/VHDL lookup tables)
fprintf('\nCoefficienti del filtro FIR:\n');
fprintf('%f, ', fir_coeffs);
fprintf('\n');


% 4. Frequency Response Calculation for the Compensating FIR Filter
fs_audio = 48000;
[h_filter, w_filter] = freqz(fir_coeffs, 1, 1024);
f_axis = (w_filter/pi) * (fs_audio/2); % Convert normalized
```

```
frequency vector to Hz


% PLOT GENERATION


figure('Name', 'Filtro FIR Compensazione', 'Color', 'w');
plot(f_axis/1000, 20*log10(abs(h_filter)), 'g', 'LineWidth', 2);
grid on;
set(gca, 'GridColor', [0.8 0.8 0.8], 'GridAlpha', 0.5); % Soft gray
grid lines
set(gca, 'Color', 'w'); % White plot area background
set(gca, 'XColor', 'k', 'YColor', 'k'); % Black border colors
set(gca, 'FontWeight', 'bold', 'FontSize', 11); % Bold axes ticks
set(gca, 'GridColor', 'k', 'GridAlpha', 0.15); % Transparent dark
grid lines


% X-axis label and styling
xlabel('Frequency (kHz)', 'FontSize', 12, 'Color', 'k',
'FontWeight', 'bold');
ylabel('Magnitude Normalized (dB)', 'FontSize', 12, 'Color', 'k',
'FontWeight', 'bold');
title('Compensating FIR filter response in frequency domain',
'Color', 'k');
xline(20, 'r--', 'Compensation limit (20 kHz)',
'LabelVerticalAlignment', 'bottom', 'FontWeight', 'bold');
xline(24, 'r--', 'cancel all above 24 khz',
'LabelVerticalAlignment', 'bottom', 'FontWeight', 'bold');
%yline(5.2734, 'r--', 'perdita circa 5 db', 'LineWidth', 1.5);
yline(3, 'r--', 'Compesation at 3db ', 'LineWidth', 1.5);
xline(15.28,'r--','frequency at 3dB is near
15.28KHz','LineWidth',1.5);
xlim([0 24]); % Constrain limits to the Nyquist boundary of the
audio rate
ylim([-80 10]); % Zoom interval to visualize passband gain and
stopband rejection


%%%%%%%%%%%%
%
==========================================================================
=====
% CASCADE SIMULATION: ANALOG INPUT SIGNAL + CIC + COMPENSATING FIR
%
```

```
================================================================================


%% 1. Generation of High-Rate Input Signal ("Analog" continuous
emulation at 2.4 MHz)
% Two components are generated to demonstrate filtering performance:
% - F1 = 10 kHz (Inside the passband: shows droop correction effect)
% - F2 = 50 kHz (Outside the passband, near the 48 kHz null: shows
anti-aliasing rejection)


f1 = 10000;  % 10 kHz signal component
f2 = 0;  % 50 kHz component (currently disabled/zeroed out)


% Enforce number of samples to be an integer multiple of R = 50
% 144,000 samples at 2.4 MHz corresponds to exactly 60 ms of signal
NumSamples = 144000;;
t_in = (0:NumSamples-1) / Fs_in; % Time vector for input signal
sampled at 2.4 MHz


% Generate the input vector as a column vector for processing
compatibility
x_in = (sin(2*pi*f1*t_in) + sin(2*pi*f2*t_in)).'; %% x_in is a
sinewave sampled at 2.4Mhz




%% 2. Processing Through CIC Filter (Decimation Stage to 48 kHz)
% The CIC filter decimates and downsamples the data from 2.4 MHz
down to 48 kHz
x_cic = step(hcic, x_in);


% Time vector calculated specifically for the decimated CIC output
rate
t_cic = (0:length(x_cic)-1) / fs_audio; % Time vector for output
signal sampled at 48 kHz
```

```
%% 3. Processing Through Compensation FIR (Equalization and Final
Rejection)
% The FIR filter runs at 48 kHz: balances out the passband drop and
filters high out-of-band aliases
x_fir = filter(fir_coeffs, 1, x_cic);




%% 4. Spectral Estimation (FFT Points Definition)
% NFFT_in: 2^16 points for fine frequency resolution on the 2.4 MHz
input data.
% NFFT_out: 2^11 points (2048) matching hardware FFT size (e.g.,
Vivado IP Core configuration).
NFFT_in = 2^16;% 2^16 punti per fare lo spettro del segnale
analogico in ingresso
NFFT_out = 2^11; %2^11 punti cioè 2048 punti che è la dimensioende
della fft nel blocco ip FFT di vivado


[X_in, f_in] = periodogram(x_in, [], NFFT_in, Fs_in);
[X_cic, f_cic] = periodogram(x_cic, [], NFFT_out, fs_audio);
[X_fir, f_fir] = periodogram(x_fir, [], NFFT_out, fs_audio);




%% 5. FIGURE 1: TIME-DOMAIN CASCADE REPRESENTATION
figure('Name', 'Effetto Cascata CIC + FIR - Tempo', 'Color', 'w');


% Subplot 1: Input stage
subplot(3,1,1);
plot(t_in*1000, x_in, 'b', 'LineWidth', 1.2); % Time converted to ms
via *1000 multiplication
grid on;
xlim([0.5 15]); % Focused display windows between 0.5 and 15 ms
title('1. Input analog signal with f=23KHz" sampled at 2.4 MHz from
Microphone', 'FontWeight', 'bold','Color', 'k');
xlabel('Time (ms)'); ylabel('Amplitude');
set(gca, 'FontWeight', 'bold', 'GridAlpha', 0.15, 'Color', 'w',
'XColor', 'k', 'YColor', 'k');


% Subplot 2: CIC output stage
```

```
subplot(3,1,2);
stem(t_cic*1000, x_cic, 'r', 'LineWidth', 1.2, 'Marker', 'o');
grid on;
xlim([0.5 15]);
title('2. Output after CIC', 'FontWeight', 'bold','Color', 'k');
xlabel('Time (ms)'); ylabel('Amplitude');
set(gca, 'FontWeight', 'bold', 'GridAlpha', 0.15, 'Color', 'w',
'XColor', 'k', 'YColor', 'k');


% Subplot 3: FIR compensated output stage
subplot(3,1,3);
stem(t_cic*1000, x_fir, 'g', 'LineWidth', 1.5, 'Marker', 'x');
grid on;
xlim([0.5 15]);
title('3.Output after FIR', 'FontWeight', 'bold','Color', 'k');
xlabel('Time (ms)'); ylabel('Amplitude');
set(gca, 'FontWeight', 'bold', 'GridAlpha', 0.15, 'Color', 'w',
'XColor', 'k', 'YColor', 'k');


figure('Name', 'Effetto Cascata CIC + FIR - Spettro
Autonormalizzato', 'Color', 'w');


% Plot overlay representing the Power Spectral Density changes
through the chain
plot(f_in/1000, 10*log10(X_in), 'b', 'LineWidth', 1.2,
'DisplayName', 'Input data (2.4 MHz)');
title('Spectral analysis', 'FontSize', 12, 'FontWeight',
'bold','Color', 'k');
xlabel('Frequency (kHz)', 'FontWeight', 'bold');
set(gca, 'FontWeight', 'bold', 'GridAlpha', 0.15, 'Color', 'w',
'XColor', 'k', 'YColor', 'k');
xlabel('Frequency (kHz)', 'FontWeight', 'bold');
ylabel('Magnitude (dB)', 'FontWeight', 'bold');
legend('Location', 'best');
grid on;
%hold on;
plot(f_cic/1000, 10*log10(X_cic), 'r', 'LineWidth', 1.2,
'DisplayName', 'after CIC (48 kHz)');
title('Spectral analysis', 'FontSize', 12, 'FontWeight',
'bold','Color', 'k');
set(gca, 'FontWeight', 'bold', 'GridAlpha', 0.15, 'Color', 'w',
'XColor', 'k', 'YColor', 'k');
```

```
xlabel('Frequency (kHz)', 'FontWeight', 'bold');
ylabel('Magnitude (dB)', 'FontWeight', 'bold');
legend('Location', 'best');
grid on;
hold on;
plot(f_fir/1000, 10*log10(X_fir), 'g', 'LineWidth', 1.5,
'DisplayName', 'after FIR (48 kHz)');
title('Spectral analysis', 'FontSize', 12, 'FontWeight',
'bold','Color', 'k');
set(gca, 'FontWeight', 'bold', 'GridAlpha', 0.15, 'Color', 'w',
'XColor', 'k', 'YColor', 'k');
xlabel('Frequency (kHz)', 'FontWeight', 'bold');
ylabel('Magnitude (dB)', 'FontWeight', 'bold');
legend('Location', 'best');
grid on;




grid on;
xlim([0 100]); % Detailed zoom up to 100 kHz to properly inspect
peaks and alias attenuation
ylim([-90 10]);
xlabel('Frequency (kHz)', 'FontWeight', 'bold');
ylabel('Magnitude (dB)', 'FontWeight', 'bold');
legend('Location', 'best');
```